

HOCHSCHULE HAMM LIPPSTADT

ADVANCED EMBEDDED SYSTEM LAB

Automatic Vehicle Accident Detection And Alert System

Md Saidur Rahman
Sahat Al Ferdous Fahim
Moshiour Rahman Prince

Department of Electronic Engineering

Supervised by
Prof. Dr. Ali Hayek

July 7, 2023

Table of Contents

1	Introduction	1
2	Concept Description	2
2.1	Key Features	2
3	Project Management	4
4	Technologies	5
4.1	Controller Units	5
4.1.1	Raspberry Pi	5
4.1.2	Arduino Uno WiFi Rev 2	5
4.2	Sensor Units	5
4.2.1	Accelerometer	5
4.2.2	GPS	6
4.3	Software Elements	6
4.3.1	MQTT Communication Protocol	6
4.3.2	Python in Our Use Case	6
5	Implementation	7
5.1	Software	7
5.1.1	System Design	7
5.1.2	Vehicle Integration	8
5.2	Hardware	9
6	Use Case	11

Chapter 1

Introduction

Welcome to the project documentation for our innovative automotive safety system aimed at providing immediate assistance during accidents. This cutting-edge system integrates advanced technology to ensure driver safety and enable prompt emergency response. With a focus on enhancing road safety, providing immediate medical attention, and enabling efficient accident management, our system aims to benefit both drivers and emergency response teams. In today's fast-paced world, road accidents are an unfortunate reality. According to statistics, millions of accidents occur each year, resulting in injuries, fatalities, and significant economic losses. Prompt medical assistance and effective accident management are crucial in minimizing the impact of these incidents and saving lives.

Our automotive safety system addresses these challenges by leveraging the power of modern technology. The core components of our system include an advanced onboard navigation system, a highly sensitive impact sensor, a specialized device called the Event Data Recorder (EDR), and a centralized control hub known as the Central Control Hub.

The onboard navigation system provides real-time GPS coordinates of the vehicle, which are periodically transmitted to the Central Control Hub. The impact sensor incorporated into our system detects collisions or accidents by accurately measuring changes in acceleration and deceleration. When an impact is detected, the EDR, installed in the vehicle, collects and processes data from the accelerometer and GPS sensors. Using a secure communication protocol, the EDR establishes a connection with the Central Control Hub, where the accident notification is received and processed. At a designated hospital, a specialized device called the Rescue Console is set up. This console subscribes to the Central Control Hub, continuously receiving real-time GPS coordinates and accident detection information. When an accident notification is received, the Rescue Console promptly alerts the hospital's emergency response team, enabling them to assess the severity of the accident and dispatch the necessary medical assistance to the crash site.

Chapter 2

Concept Description

The proposed project aims to develop an advanced car safety system that utilizes GPS technology, accelerometers, and IoT devices to enhance driver safety, provide real-time monitoring, and enable efficient emergency response in the event of an accident. The system consists of a car with built-in GPS and accelerometer sensors, an Arduino microcontroller for data collection and transmission, a Raspberry Pi acting as a central broker and server, and an additional Arduino for accident detection and emergency response coordination.

2.1 Key Features

1. **Accelerometer-based Accident Detection:** The integrated accelerometer detects sudden changes in the car's acceleration, allowing for the detection of potential accidents. When an accident is detected, the system initiates immediate emergency response procedures.
2. **MQTT-based Communication:** The Arduino collects both GPS coordinates and accelerometer data and transmits it to a remote broker using the MQTT protocol. This lightweight and reliable messaging protocol ensure efficient and secure data transfer.
3. **Raspberry Pi as a Broker:** The Raspberry Pi acts as a central broker, receiving data from the car's Arduino and serving as an intermediary for communication between devices. It facilitates the seamless exchange of information between the car, emergency response systems, and the user.
4. **Emergency Response Coordination:** The additional Arduino subscribes to the broker to receive GPS coordinates and accident detection information. When an accident is detected, it immediately alerts the emergency response team and initiates a drone dispatch to the crash site for rapid assessment and assistance.

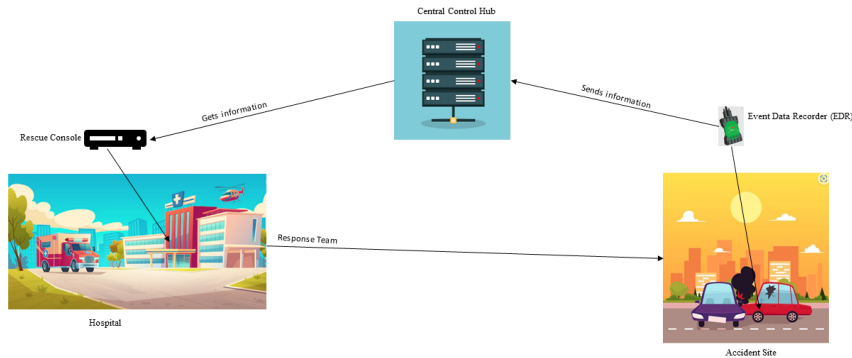


Figure 2.1: Project Concept

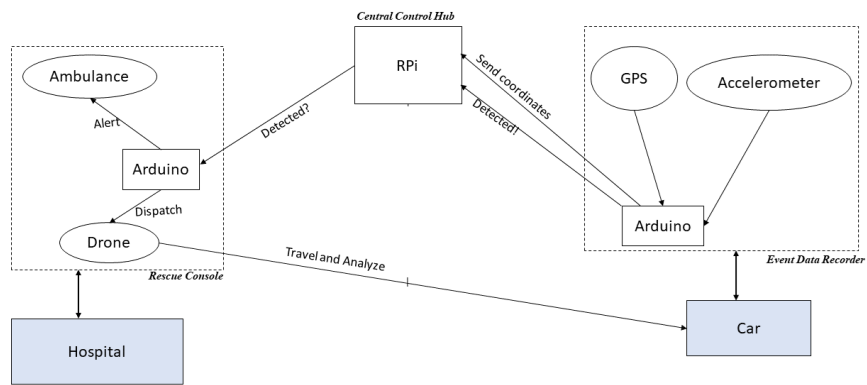


Figure 2.2: Block Diagram

The integrated car safety system presented here offers a comprehensive solution to enhance driver safety, provide real-time monitoring, and enable efficient emergency response. By leveraging GPS technology, accelerometers, and IoT devices, the system ensures prompt accident detection, immediate alerting of emergency services, and the deployment of a drone for on-site assessment. The project combines advanced hardware and software components to create a robust and professional-grade system that can potentially make a significant impact on road safety and emergency response effectiveness.

Chapter 3

Project Management

In order to effectively organize our project, we have made the decision to divide our tasks into smaller, more manageable units and assign them to individual team members. The purpose of this approach is to ensure clear accountability and promote efficient progress. Below is a table that outlines the tasks required for the project, along with the corresponding responsible team member for each task:

Name	Tasks
Sahat al Ferdous Fahim	1. Setting up environment. 2. Designing system architecture. 3. Drawing Schematic. 4. MQTT implementation.
Saidur Rahman	1. Sensor Node Implementation 2. Get accelerometer Data. 3. Get GPS Data.
Moshiour Rahman Prince	1. Creating Hardware Setup. 2. Triggering actuators.

By assigning specific tasks to individual team members, we aim to foster a sense of ownership and ensure that each aspect of the project receives dedicated attention. This organized approach helped us effectively track progress and achieve our collective goals.

Chapter 4

Technologies

The system will utilize Raspberry Pi, Arduino Uno WIFI, MQTT protocol, an accelerometer, and a GPS module to detect car crashes and alert emergency services. The Raspberry Pi will act as the MQTT broker, while the Arduinos will serve as sensor and actuator nodes. The system will also publish data to a web platform for visualization and storage.

4.1 Controller Units

4.1.1 Raspberry Pi

The Raspberry Pi is a small, affordable, single-board computer that can run various operating systems, including Linux. In this project, the Raspberry Pi will act as the MQTT broker, managing communication between the Arduino nodes and the web platform. It will also process and store data, as well as provide a user interface for visualization. The Raspberry Pi's low cost, small form factor, and powerful processing capabilities make it an ideal choice for IoT projects like this car accident alert system.

4.1.2 Arduino Uno WiFi Rev 2

The Arduino Uno WiFi Rev 2 is a microcontroller board based on the ATmega4809 microcontroller, featuring built-in WiFi capabilities. In this project, two Arduino Uno WiFi Rev 2 boards will be used as sensor and actuator nodes. The sensor node will be connected to the accelerometer and GPS module, while the actuator node will control a buzzer and an LED to mimic an ambulance. The Arduino Uno WiFi Rev 2's ease of use, built-in WiFi, and compatibility with various sensors and actuators make it a suitable choice for this IoT project.

4.2 Sensor Units

4.2.1 Accelerometer

An accelerometer is an electromechanical device that measures acceleration forces. In this project, it will be used to detect sudden changes in a car's motion, which may indicate a crash. Accelerometers

work by measuring the force exerted on a mass inside the device. When the car experiences a sudden change in motion, the mass inside the accelerometer moves, generating an electrical signal proportional to the acceleration. This signal is then processed by the Arduino to determine if a crash has occurred.

4.2.2 GPS

The Global Positioning System (GPS) is a satellite-based navigation system that provides accurate location and time information. In this project, a GPS module will be used to determine the car's location in the event of a crash. The GPS module receives signals from multiple satellites and calculates the car's position by triangulating the time it takes for the signals to reach the receiver. This information is then sent to the Raspberry Pi for further processing and visualization on the web platform.

4.3 Software Elements

4.3.1 MQTT Communication Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight, publish-subscribe messaging protocol designed for low-bandwidth, high-latency, or unreliable networks. In this project, MQTT will be used to facilitate communication between the Raspberry Pi (broker) and the Arduino nodes (clients). The sensor node will publish data to the broker, which will then forward the data to the actuator node and the web platform. MQTT's lightweight nature and efficient use of network resources make it an ideal choice for IoT applications like this car accident alert system prototype.

4.3.2 Python in Our Use Case

Python is a versatile, high-level programming language that is widely used in various applications, including IoT. In this project, Python will be used to program the Raspberry Pi, which will act as the MQTT broker and handle data processing, storage, and visualization. Python's extensive library support, ease of use, and readability make it an excellent choice for implementing the Raspberry Pi's functionality in this project.

Chapter 5

Implementation

5.1 Software

5.1.1 System Design

Our innovative automotive safety system is designed to provide immediate assistance during accidents by integrating cutting-edge technology and establishing a robust system architecture. Let's explore the components and their functionalities shown in Fig.5.1 in more detail.

- Event Data Recorder (EDR): The Event Data Recorder plays a crucial role in our system. It collects sensor data from the car and publishes it to a topic named Car_ID/EDR/Data. This data includes essential information captured by the car's sensors. The EDR also detects crashes, and upon crash detection, it publishes the car's license number (car_id) to a separate topic called car_info.

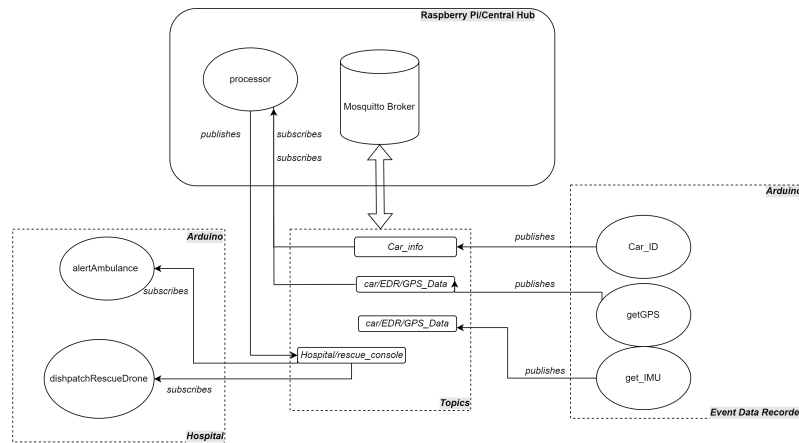


Figure 5.1: System Architecture

- Central-Hub: Our system leverages a central hub (RPi) for hosting the MQTT broker. Addi-

tionally, another client running in the raspberry pi (RPi) subscribes to the car_info topic. Once a unique car ID is received on this topic, the crash detector application responds accordingly. It creates a topic with the received car ID and subscribes to it to receive relevant information and forwards them to the rescue console.

- **Rescue Console:** The Rescue Console is a specialized device located within the designated hospital. Its purpose is to receive data related to accidents on a specific topic. When an accident occurs and relevant information is published on the accident topic, the emergency response team within the hospital takes immediate action based on the received data.

Our automotive safety system operates as an event-driven system, where each event prompts a specific state to initiate appropriate actions. The complete state machine behavior of the system is depicted in Fig. 5.2, providing a comprehensive overview of its functionality.

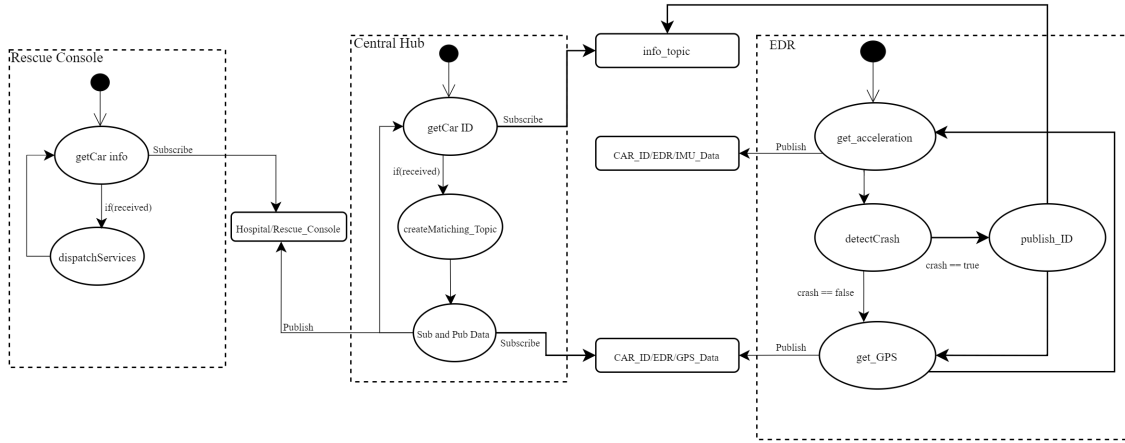


Figure 5.2: State Machine

5.1.2 Vehicle Integration

The topic namespace plays a crucial role in seamlessly integrating an infinite number of vehicles into the network. To ensure a robust and secure integration, each vehicle is only allowed to publish on topics starting with a unique identifier.

Leveraging the uniqueness of license plates, we utilize them as the unique identifiers for each vehicle. Fig. 5.3 shows a comprehensive overview of vehicle integration. One specific topic, `Car_info`, is accessible to all vehicles connected to the network, but it is reserved exclusively for publishing the unique identifier of the car, which in this case is the license plate number. The central hub, acting as the control center, receives the unique identifier of the crashed car through this topic and promptly takes action. It subscribes to topics relevant to that specific car, enabling the retrieval of pertinent information, including GPS coordinates.

Upon receiving the relevant information, the central hub forwards it to the hospital's rescue console. In a real-life implementation, this could be sent to a nearby hospital that is strategically located in close proximity to the affected car. Armed with the received information, the hospital's authorities can swiftly initiate the necessary actions and response measures. This approach ensures

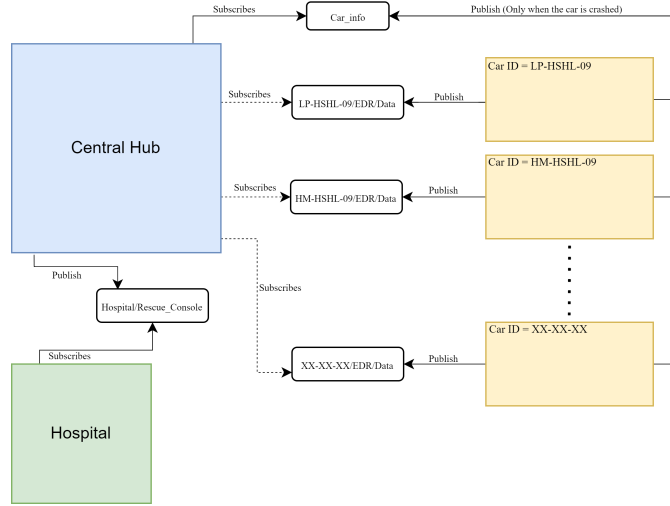


Figure 5.3: Seamless integration of an unlimited number of vehicles

rapid and targeted emergency response, minimizing response times and enhancing overall accident management effectiveness.

5.2 Hardware

This hardware schematic in Fig. 5.4 depicts a networked system comprising three Wi-Fi-connected Arduino boards and a Raspberry Pi. Two of the Arduino boards are equipped with GPS sensors, while the third board features a buzzer, servo, and LED. All Arduino boards have built-in IMUs, enhancing motion-sensing capabilities. The WiFi connection enables MQTT telemetry data transfer and control commands between the devices.

The source code for the complete implementation is available at [1].

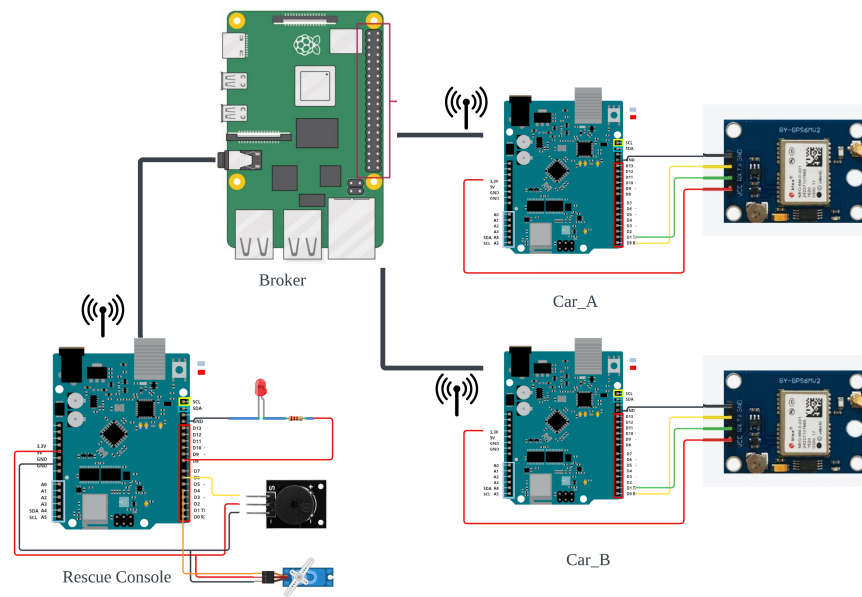


Figure 5.4: Schematic

Chapter 6

Use Case

The use case scenarios of our system can be divided into three parts, Hardware setup, Software setup, and finally User interaction.

Hardware Setup: Before flashing the firmware into the Arduino it is important to make sure all the pins are connected according to the software. The connection of both Arduino vehicle and rescue console is given in the table below:

Name	Component	Pin
Rescue Console(Arduino)	1. Servo. 2. Buzzer. 3. LED.	10 8 7
EDR (Arduino)	1. GPS Module. 2. IMU.	rx→tx; tx→rx Builtin
Central Hub (RPi)	No component.	No pins

Software Setup: We have use a VScode extension called platformIO for development. It is recommended to open our project using platformIO for editing and flashing our source code. For our system to work, all the devices must be under a same wifi network. Wifi SSID and Password for arduinos if located in the following directory of the project file, <Project_name>/lib/Secrets/super_secret.h.

Once the Borker is connected to a new network, it gets a new IP address. The IP address of all the application also need to be changed, this includes processor.py in RPi, and all the arduino clients. Once everything is setup, flash the arduinos and run the processor.py in raspberry pi. Our system is ready for user interaction.

User Interaction: Since we are using the builtin accelerometer to detect the accident. A user needs to shake the arduino (vehicle side) for the accident to take place. If an accident take place the other arduino (hospital) will trigger a response e.g, blink LED, run servo, and turn on buzzer. A video of the use case is available with this [link](#)

References

- [1] Moshiour Rahman Prince Sahat al Ferdous Fahim, Saidur Rahman. The a-team. https://github.com/Sahat-fahim/The_A-Team_HSHL/tree/main, 2023.